Implementation Document

for

SkipQ

Version <1.1>

Prepared by

Group #: 9 **Group Name:** Commit and Conquer

Surya Prakash Pingali	241067	suryap24@iitk.ac.in
Usnish Paul	241113	usnishp24@iitk.ac.in
Aayush Gajeshwar	240024	aayushk24@iitk.ac.in
Harpuneet Singh	240426	harpuneets24@iitk.ac.in
Sourabh Sankhala	241036	sourabhs24@iitk.ac.in
Pratham Sachin Todkar	240780	prathams24@iitk.ac.in
Jashan Mittal	240488	jashanm24@iitk.ac.in
Aarya Bhatt	240162	aaryajb24@iitk.ac.in
Jatin Toshniwal	240489	jatin24@iitk.ac.in

Course: CS253

Mentor TA: *Shaikh Imran*

Date: 27-03-2026

Course: CS253

Mentor TA: *Shaikh Imran*

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

APPENDIX A - GROUP LOG 4

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
v1.0	Aayush Gajeshwar Harpuneet Singh Surya Prakash Pingali Aarya Bhatt Jashan Mittal Jarapla Ashish Pamar Usnish Paul Sourabh Sankhala Pratham Sachin Todkar	First Version of the Implementation Document	00/00/00

1. Implementation Details

Live Deployment:

Customer / Owner Login - *http://172.27.16.252:3000*

Admin Login - *http://172.27.16.252:3000/admin/login*

Backend API Root - *http://172.27.16.252:8000/api/*

SkipQ is a full-stack canteen automation and food ordering platform that leverages a modern web technology stack React.js, Django, and PostgreSQL, to deliver a fast, scalable, and user-friendly solution. The system seamlessly connects customers, canteen managers, and administrators through a centralized platform, simplifying the entire workflow from browsing menus and placing orders to real-time tracking and payment management.

PostgreSQL (with SQLite for development) serves as the system's database, enabling efficient storage and retrieval of user profiles, canteen data, menu items, orders, and transaction records. Its structured relational model ensures data consistency and integrity, which is critical for handling concurrent users, financial transactions, and order histories without anomalies.

Django and Django REST Framework form the backbone of the server-side implementation. Django manages core functionalities such as authentication, database interactions, and security, while Django REST Framework provides a clean and scalable interface for building RESTful APIs. This combination ensures secure communication between the frontend and backend, efficient request handling, and smooth integration with future services such as payment gateways and machine learning modules.

On the frontend, React.js powers a dynamic and responsive user interface, allowing users to navigate seamlessly across pages for browsing canteens, selecting food items, managing carts, and tracking orders. Its component-based architecture promotes reusability, maintainability, and efficient state management, ensuring a consistent and engaging user experience across devices.

Overall, SkipQ's implementation demonstrates how a modern full-stack architecture can be used to build a robust and scalable campus service platform. By combining Django's powerful backend capabilities with React's flexible frontend architecture and a reliable relational database system, SkipQ delivers an intuitive, efficient, and future-ready solution for optimizing canteen operations and reducing waiting times.

Programming Languages, Framework, and Libraries

Programming Language

SkipQ has been primarily developed using JavaScript and Python. It follows a layered client–server architecture, ensuring clear separation between frontend, backend, and data layers, which improves scalability and maintainability.

For the Backend, we have used:

1. **Python (Django Framework):**
   The backend of SkipQ is implemented using Python with Django and Django REST Framework. Compared to traditional languages like C++ or Java, Python offers faster development, cleaner syntax, and a vast ecosystem of libraries. Django provides built-in features such as authentication, ORM, and security, while Django REST Framework enables efficient API development. This combination allows for rapid development, secure handling of requests, and easy scalability of backend services.

For the Frontend, we have used:

1. **HTML (HyperText Markup Language):**
   HTML is used to structure the web pages of SkipQ. It forms the backbone of all user interfaces and ensures compatibility across all modern web browsers.
2. **CSS (Cascading Style Sheets) with Tailwind CSS:**
   CSS is used for styling the application and ensuring a consistent and responsive design. Tailwind CSS, a utility-first framework, enhances development speed and maintainability by allowing rapid UI creation with reusable styling classes.
3. **JavaScript (with React.js):**
   JavaScript is used to build dynamic and interactive user interfaces. React.js enables component-based development, making the application modular and scalable. It also improves performance through efficient rendering using the virtual DOM. JavaScript executes directly in the browser, ensuring fast user interactions and reducing dependency on server-side rendering.

For the DBMS (Database Management System), we have used:

1. **SQLite (Development) and PostgreSQL (Production Target):**
   SQLite is used during development due to its lightweight nature and ease of setup. For production, PostgreSQL is preferred because of its high performance, reliability, and ability to handle concurrent users efficiently. It supports structured relational data, ensuring consistency in managing users, orders, transactions, and canteen data, which is critical for a system like SkipQ.

Frameworks

Build System:

We have used **Vite** as the primary build tool along with modern JavaScript tooling due to the following benefits:

1. **Fast Development Environment:**
   Vite provides extremely fast startup times and instant Hot Module Replacement (HMR), allowing developers to see changes in real time without reloading the entire application.
2. **Efficient Dependency Handling:**
   Vite uses native ES modules and optimized bundling, enabling faster dependency resolution and reducing build times significantly compared to traditional bundlers.
3. **Optimized Production Builds:**
   It generates highly optimized production builds with efficient code splitting, improving application performance and load times.
4. **Simplified Configuration:**
   Vite requires minimal configuration, making it easy to set up and maintain, which accelerates the development process.

Django:

We have used **Django** as the primary backend framework for our web application due to its robustness and scalability.

1. **Rapid Development:**
   Django provides built-in features like ORM, authentication, and admin panels, reducing development time and eliminating repetitive coding.
2. **Security Features:**
   It includes protections against common vulnerabilities such as SQL injection, CSRF, and XSS, ensuring secure handling of user data and transactions.
3. **Scalable Architecture:**
   Django supports scalable application design, making it suitable for handling increasing numbers of users and requests.
4. **Clean and Maintainable Codebase:**
   Its structured approach promotes clean coding practices and modular design, improving long-term maintainability.

Django REST Framework (DRF):

We have used **Django REST Framework** for building RESTful APIs.

1. **Simplified API Development:**
   DRF provides serializers and viewsets that simplify the creation of APIs for communication between frontend and backend.
2. **Authentication and Permissions:**
   It offers built-in support for authentication and role-based access control, ensuring secure API access.

3. **Flexibility and Extensibility:**
   DRF allows easy customization of API behavior, making it adaptable to complex business logic.
4. **Standardized Communication:**
   It ensures consistent JSON-based communication between frontend and backend systems.

React.js:

We have used **React.js** as the frontend framework due to its flexibility and performance.

1. **Component-Based Architecture:**
   React enables the creation of reusable UI components, improving code organization and reducing redundancy.
2. **Efficient Rendering (Virtual DOM):**
   React uses a virtual DOM to update only the necessary parts of the UI, improving performance.
3. **Unidirectional Data Flow:**
   Ensures predictable data handling, making debugging and maintenance easier.
4. **Strong Ecosystem Support:**
   React has a large community and extensive library support, enabling faster development and problem-solving.

React Router:

We have used **React Router** for handling navigation in the application.

1. **Dynamic Routing:**
   Allows seamless navigation between different pages without reloading the application.
2. **Nested Routing Support:**
   Enables structured and hierarchical routing for complex UI flows.
3. **Improved User Experience:**
   Provides smooth transitions between pages, enhancing usability.

Libraries

We have used a set of lightweight and focused libraries in SkipQ to improve user interface design, state handling, and overall development efficiency.

React.js:

React.js is the primary library used for building the frontend of SkipQ.

1. **Component-Based Architecture** – Allows breaking the UI into reusable components such as MenuCard, CartSidebar, and Header, making the code modular and easier to maintain.
2. **Efficient Rendering** – React updates only the required parts of the UI, which improves performance during frequent updates like cart changes or order tracking.
3. **State Management Integration** – Works seamlessly with React Context API used in the project for managing cart, wallet, and theme states.

Tailwind CSS:

Tailwind CSS is used for styling all frontend components.

1. **Utility-First Approach** – Styling is applied directly using utility classes, reducing the need for separate CSS files.
2. **Consistency Across UI** – Ensures uniform design across pages like menu browsing, checkout, and dashboards.
3. **Responsive Design** – Helps in building mobile-friendly interfaces, which is important since users often access the system from phones.

Radix UI:

Radix UI primitives are used for building interactive UI components.

1. **Accessible Components** – Provides pre-built accessible elements like dialogs, dropdowns, and inputs.
2. **Reusable UI Elements** – Wrapped into custom components inside `components/ui/` for consistent usage across the application.
3. **Improved User Experience** – Ensures smooth interactions in elements like modals and forms.

React Hook Form:

React Hook Form is used for managing forms such as login, registration, and checkout.

1. **Efficient Form Handling** – Reduces unnecessary re-renders compared to traditional form handling.
2. **Validation Support** – Easily integrates validation for user inputs.
3. **Cleaner Code Structure** – Simplifies handling of form state and submission logic.

Sonner (Toast Notifications):

Sonner is used for showing notifications to users.

1. **User Feedback** – Displays messages for actions like adding items to cart, successful payments, or errors.
2. **Non-Intrusive UI** – Notifications appear without interrupting the user flow.

Lucide React:

Lucide React is used for icons in the UI.

1. **Lightweight Icons** – Provides simple and clean icons for buttons and navigation.
2. **Consistent Design** – Maintains uniform icon styling across the application.

clsx & tailwind-merge:

These libraries are used for handling dynamic class names in Tailwind CSS.

1. **Conditional Styling** – Helps apply styles based on component states.
2. **Cleaner Code** – Avoids long and complex class name conditions.

Pillow (Backend Library):

Pillow is used in the backend for image processing.

1. **Image Conversion** – Converts uploaded images into a standard format (JPEG).
2. **Optimization** – Reduces file size for faster loading.
3. **Format Handling** – Supports multiple input formats.

Authentication

SkipQ implements a secure authentication system with the following key components:

1. **Password Protection:**
   User passwords are securely stored using Django's built-in authentication system, which applies hashing before saving them in the database. This ensures that sensitive credentials are never stored in plain text, protecting user data in case of any potential breach.
2. **Session-Based Authentication:**
   SkipQ uses session-based authentication instead of token-based systems. Upon successful login, a session is created and maintained using secure cookies. This allows the server to track authenticated users across requests while ensuring controlled access to protected routes.
3. **OTP-Based Email Verification:**
   The system implements an OTP (One-Time Password) verification mechanism during registration and password reset. OTPs are generated and stored temporarily, and users must verify them to complete account creation or update credentials. This adds an extra layer of security and ensures that only valid users can access the platform.
4. **Role-Based Access Control:**
   Users are assigned specific roles such as **Customer, Canteen Manager, and Admin**. Access to different parts of the system is controlled based on these roles, ensuring that users can only perform actions permitted for their role.
5. **Secure API Access:**
   All protected API endpoints require authentication via active sessions. Unauthorized access attempts are restricted, ensuring that sensitive operations like order placement, wallet access, and menu management are securely handled.

In conclusion, SkipQ leverages a well-structured combination of modern web technologies to deliver a scalable, efficient, and user-centric canteen management platform. The integration of React.js for the frontend and Django with Django REST Framework for the backend ensures a clear separation of concerns while maintaining smooth communication across the system. JavaScript, as the interactive layer on the client side, enables dynamic and responsive user interfaces, while Python powers the backend with robust logic, security, and efficient data handling.

The use of a relational database system such as PostgreSQL ensures data consistency and integrity across critical components like user profiles, orders, transactions, and canteen operations. Django's built-in ORM and authentication mechanisms simplify database interactions and enhance security, while the RESTful API architecture enables seamless integration between frontend and backend components.

Additional libraries such as Tailwind CSS, React Hook Form, and Radix UI contribute to a clean, responsive, and accessible user interface, improving the overall user experience. The implementation of session-based authentication, OTP verification, and role-based access control ensures secure and controlled access to system functionalities.

Overall, SkipQ demonstrates how a thoughtfully designed full-stack architecture can effectively address real-world challenges such as reducing waiting times and improving canteen efficiency. By combining performance-oriented technologies with a modular and extensible design, the system provides a reliable and future-ready solution for campus-wide food ordering and management.

2. Codebase

GitHub Repository: SkipQ

Code Structure:

The above reference details the GitHub repository of SkipQ, which contains all the source code for our comprehensive canteen automation and queue management platform.

Version Control Strategy

The repository employs a structured, dual-branch version control system to isolate the production environment from active development. The architecture utilizes a primary `main` branch and a secondary `dev` (development) branch.

All active modifications to the codebase—including feature implementation, bug resolutions, and structural refactoring—are executed exclusively within the `dev` branch. The `main` branch is strictly designated for stable, production-ready code. Code is merged from `dev` into `main` only after testing and validation. This systematic segregation ensures that any systemic instability or critical failures introduced during the development remain contained within the `dev` environment, thereby preserving the structural integrity of the `main` branch.

The main branch of the project repository is primarily composed of 3 main parts:

- **Frontend**
- **Backend**
- **Documents**

Overall Structure of the repository for SkipQ:

```
|--SkipQ/
  |-- Backend/ // Backend Directory (Django)
  |-- Frontend/ // Frontend Directory (React/Vite)
  |-- .gitignore // Git ignore configuration
  |-- README.md
  |-- start.sh
```

Frontend

Inside the frontend folder, you'll find the React app. Here's how it's broken down:

- src: Where all the actual code lives:
- components: The reusable UI bits (Header, Footer, CartSidebar).
- pages: The main screens users see (like Checkout or MenuBrowsing).
- context: React context for global state.
- styles: Global CSS and utility classes.
- index.css: The main stylesheet.
- App.tsx & main.tsx: The React mounting point and root component.

- vite.config.ts: Vite's build settings.
- package.json: Node dependencies and scripts.

The frontend is a fairly standard React single-page app using React Router. We group pages and components by feature—so if you're looking for the canteen menu or the user profile, they drop right into their own focused folders.

Frontend Structure

```
|-- Frontend/
   |-- src/                          // Application source code
      |-- api/                       // API service layers for backend communication
      |-- assets/                    // Static assets like icons and images
      |-- components/                // Reusable UI elements
         |-- AddToCartButton.tsx     // Button for adding items to cart
         |-- CartSidebar.tsx         // Slide-out cart view
         |-- Footer.tsx              // Standard page footer
         |-- Header.tsx              // Global top navigation bar
         |-- ProtectedRoute.tsx      // Auth guard for protected routes
         |-- TrackOrderSidebar.tsx   // Order tracking slide-out view
         |-- figma/                  // Design reference components
         |-- ui/                     // Common UI primitives
      |-- context/                   // React Context providers for state
      |-- data/                      // Static data resources or mock data
      |-- imports/                   // Shared import utilities
      |-- pages/                     // Main application views
         |-- AdminLogin.tsx          // Administrator authentication page
         |-- AdminPanel.tsx          // Global system administration dashboard
         |-- CakeManagement.tsx      // Interface for managing cake inventory
         |-- CakeReservation.tsx     // User interface for pre-ordering cakes
         |-- CanteenRegistration.tsx // Onboarding for new canteens
         |-- Cart.tsx                // Full cart review page
         |-- Checkout.tsx            // Payment and checkout flow
         |-- ForgotPassword.tsx      // Account recovery interface
         |-- HostelSelection.tsx     // User hostel selection screen
         |-- MenuBrowsing.tsx        // Food menu browsing interface
         |-- MenuManagement.tsx      // Interface for canteen staff to manage items
         |-- OrderConfirmation.tsx   // Post-checkout success screen
         |-- OwnerAccount.tsx        // Dashboard for system administrators
         |-- OwnerDashboard.tsx      // Main overview for canteen owners
         |-- OwnerRegistration.tsx   // Onboarding for canteen owners
         |-- PaymentResult.tsx       // Post-payment status display
         |-- ScheduleManagement.tsx  // Handling opening/closing schedules
         |-- SearchResults.tsx       // Display search queries
         |-- SetWalletPin.tsx        // Security setup for user wallet
         |-- Statistics.tsx          // Data analytics and reporting views
         |-- TrackOrders.tsx         // User order history and status
         |-- UnifiedLogin.tsx        // Main authentication screen for all users
```

```
        |-- UserProfile.tsx          // User account management
        |-- VerifyWalletPin.tsx      // PIN verification for transactions
        |-- WalletPage.tsx           // Balance and transaction history
    |-- styles/                      // Global application styling definitions
    |-- utils/                       // Internal utility functions
    |-- App.tsx                      // Main application component and router setup
    |-- index.css                    // Global root styles
    |-- main.tsx                     // Web app rendering entry point
    |-- routes.tsx                   // Router configuration
    |-- types.ts                     // Global TypeScript type definitions
  |-- package.json                   // Project dependencies and build scripts
  |-- vite.config.ts                 // Vite bundler configuration
```

Backend

The backend is a standard Django setup. The core logic lives in `apps`:

- apps: The separate Django apps handling business logic. Each app contains standard Django files like `models.py`, `views.py`, and `urls.py`.
- administration: Custom admin tools.
- cakes: Cake ordering and custom requests.
- canteens: Menu and canteen operations.
- orders: Cart processing.
- users: Auth and profiles.
- config: The actual Django backbone (settings.py, urls.py).
- tests: Backend test suites.
- manage.py: Django's built-in CLI for running servers and migrations.
- requirements.txt: Python dependencies.
- README.md: Project docs.

Backend Structure

```
|-- Backend/
    |-- Database/                  // Core database schema and scripts
    |-- apps/                      // Core logic broken into Django modules
        |-- administration/        // Global administrative functions
        |-- cakes/                 // Cake and custom pre-order feature
            |-- migrations/        // Database migration history
            |-- services/          // Core business logic services
            |-- admin.py           // Admin panel configuration for cakes
            |-- apps.py            // App registry configuration
            |-- models.py          // Database models for cakes and toppings
            |-- serializers.py     // Data serialization for the API
            |-- tests.py           // Unit tests for cake logic
            |-- urls.py            // API route mapping for cakes
            |-- views.py           // Business logic and request handling
```

```
|-- canteens/              // Canteen and general food menu logic
    |-- migrations/        // Database migration history
    |-- services/          // Core business logic services
    |-- admin.py           // Admin panel configuration for canteens
    |-- apps.py            // App registry configuration
    |-- models.py          // Database models for food items
    |-- serializers.py     // Data serialization for the API
    |-- tests.py           // Unit tests for canteens logic
    |-- urls.py            // API route mapping for menus
    |-- views.py           // Business logic and request handling
|-- orders/                // Order placement and tracking logic
    |-- migrations/        // Database migration history
    |-- services/          // Core business logic services
    |-- admin.py           // Admin panel for reviewing orders
    |-- apps.py            // App registry configuration
    |-- models.py          // Database models for transactions
    |-- serializers.py     // Data serialization for the API
    |-- tests.py           // Unit tests for order logic
    |-- urls.py            // API route mapping for orders
    |-- views.py           // Business logic and request handling
|-- users/                 // Authentication and user profiles
    |-- migrations/        // Database migration history
    |-- services/          // Core business logic services
    |-- admin.py           // Admin panel for user management
    |-- apps.py            // App registry configuration
    |-- models.py          // Custom user database models
    |-- serializers.py     // Data serialization for user APIs
    |-- tests.py           // Unit tests for auth logic
    |-- urls.py            // API route mapping for users
    |-- views.py           // Business logic and request handling
|-- config/                // Root Django configuration
    |-- __init__.py        // Package initialization
    |-- asgi.py            // Async server gateway interface
    |-- authentication.py  // Core auth system overrides
    |-- settings.py        // Main project environment settings
    |-- urls.py            // Root router directing traffic to apps
    |-- wsgi.py            // Web server gateway interface
|-- tests/                 // General integration test suite
|-- manage.py              // Execution script for Django commands
|-- requirements.txt       // Pip dependencies definition
|-- README.md              // Backend setup instructions
```

Database

The database uses a straightforward relational Postgres setup. We preferred this over NoSQL for an application like this—we're dealing with user wallets, orders, and inventory. We need real tables with hard constraints.

Database Structure

```
|-- Database/
  |-- database.py        // Handles the actual connection pooling to Postgres
  |-- queries.py         // Helper functions to execute raw SQL safely
  |-- schema.sql         // The master file where all our tables (Users, Orders, Dishes)
                              are      defined
  |-- seed.py            // Script to inject dummy data so we can actually test   the app
  |-- dbsetup.md         // Quick-start guide for getting the local DB running
```

Architecture

SkipQ runs on a decoupled backend-frontend setup. It basically mirrors a traditional MVC pattern, just stretched across two separate environments. It splits the concerns so the API and UI code aren't constantly stepping on each other.

Architectural Components

Model: Django ORM schemas in the apps/*/models.py directories lock down the data structure:

- People models (Users, Admins) handle authentication and wallet balances
- Inventory models (Canteens, Dishes, Favorite_Dishes) track what's available to eat
- Transaction models (Orders, Order_Items, Payments) handle the actual business logic of buying food

View: A React and Vite single-page app renders the user interface:

- We organize everything by feature in src/pages. If you need the cake reservation system, it's exactly where you expect it in CakeReservation.tsx
- The UI relies on shared primitives in src/components/ui/ so we don't end up with ten different button styles across the app
- We lean on React Context providers for global state, which keeps us from prop-drilling the user's cart through twenty layers of components

Controller: Django views and serializers act as the traffic cops:

- Each Django app has specific urls.py files mapping API endpoints to their functions
- Serializers translate incoming JSON into Python objects, validating the data before it ever touches the database
- A custom authentication layer in config/authentication.py overrides the standard Django auth to handle our specific login flow
- Key Technical Implementations of the Architecture

- The Wallet System: We baked a wallet balance directly into the User model. It lets students pay with internal currency, bypassing external payment gateways for every single coffee or samosa.
- Modular Backend: Breaking Django into specific apps like cakes, canteens, and orders keeps the codebase from turning into a massive monolith. If the custom cake ordering logic breaks, the main canteen menu stays up.
- Audit Logging: Every administrative action gets dumped into the Admin_Activity_Logs table. It's a lifesaver when you need to figure out who accidentally deleted a menu item at 2 AM.
- Data Integrity: The database schema makes heavy use of Postgres CHECK constraints (like making sure emails end in @iitk.ac.in and quantities are above zero). We don't just trust the application layer to send good data.

Key Technical Implementations of the Architecture

● The Wallet System: We baked a wallet balance directly into the User model. It lets students pay with internal currency, bypassing external payment gateways for every single coffee or samosa.

● Modular Backend: Breaking Django into specific apps like cakes, canteens, and orders keeps the codebase from turning into a massive monolith. If the custom cake ordering logic breaks, the main canteen menu stays up.

● Audit Logging: Every administrative action gets dumped into the Admin_Activity_Logs table. It's a lifesaver when you need to figure out who accidentally deleted a menu item at 2 AM.

● Data Integrity: The database schema makes heavy use of Postgres CHECK constraints (like making sure emails end in @iitk.ac.in and quantities are above zero). We don't just trust the application layer to send good data.

API endpoints

The following is a detailed summary of all API calls in the application. All endpoints are under the `/api/` namespace.

Authentication

a. Register User

URL: `/api/auth/register/`

Method: POST

Data: `{ email, password, role, name (optional), phone (optional) }`

Status:

```

If successful:
```

```
  200 Success (OTP sent to email)
else:
  400 User already exists / Invalid email domain
  500 Server error
```

b. Verify OTP

URL: `/api/auth/verify-otp/`
Method: POST
Data: `{ email, otp }`
Status:
```
If successful:
  201 User successfully registered (returns user data + session)
  200 Manager registration pending admin approval
else:
  400 Invalid OTP / Expired OTP
  500 Server error
```

c. Login

URL: `/api/auth/login/`
Method: POST
Data: `{ email, password, remember_me }`
Status:
```
If successful:
  200 Login successful (returns user data, role, has_wallet_pin)
else:
  400 Wrong password / User not found / Account suspended
  500 Server error
```

d. Logout

URL: `/api/auth/logout/`
Method: POST

Data: None (session-based)
Status:
```
If successful:
  200 Logged out successfully
else:
  401 Not authenticated
```

e. Forgot Password

URL: `/api/auth/forgot-password/`
Method: POST
Data: `{ email }`
Status:
```
If successful:
  200 OTP sent to your email
else:
  400 Email not found
  500 Server error
```

f. Verify Forgot Password OTP

URL: `/api/auth/verify-forgot-password-otp/`
Method: POST
Data: `{ email, otp }`
Status:
```
If successful:
  200 OTP verified
else:
  400 Invalid OTP / Email and OTP are required
  500 Server error
```

g. Reset Password

URL: `/api/auth/reset-password/`
Method: POST
Data: `{ email, otp, new_password }`
Status:
```
If successful:
  200 Password reset successful
else:
  400 Invalid OTP / Password too short
  500 Server error
```

Users

a. View / Update Profile

URL: `/api/users/profile/`
Method: GET / PATCH
Data (PATCH): `{ name, phone }` (partial updates allowed)
Status:
```
If successful:
  200 Returns profile data (includes has_wallet_pin)
else:
  401 Not authenticated
  500 Server error
```

b. View Wallet Balance

URL: `/api/users/wallet/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns { balance }
```

else:
  400 No wallet for this role
  401 Not authenticated
```

**c. Add Funds to Wallet**

URL: `/api/users/wallet/add-funds/`
Method: POST
Data: `{ amount }`
Status:
```
If successful:
  200 Funds added (returns new balance)
else:
  400 Invalid amount
  403 Only customers can add funds
  401 Not authenticated
```

**d. Set Wallet PIN**

URL: `/api/users/wallet/set-pin/`
Method: POST
Data: `{ pin }` (4–6 characters)
Status:
```
If successful:
  200 Wallet PIN updated
else:
  400 Invalid PIN / No wallet for this role
  401 Not authenticated
```

---

## Canteens
```

a. List All Active Canteens

URL: `/api/canteens/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of active canteens
```

b. Canteen Detail

URL: `/api/canteens/<canteen_id>/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns canteen details
else:
  404 Canteen not found
```

c. Estimated Wait Time

URL: `/api/canteens/<canteen_id>/wait-time/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns { wait_time_minutes }
else:
  404 Canteen not found
```

d. Lead Time Config

URL: `/api/canteens/<canteen_id>/lead-time/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns { canteen_id, lead_time_hours }
else:
  404 Canteen not found
```

e. Register Canteen

URL: `/api/canteens/register/`
Method: POST (multipart/form-data)
Data:
```
Text fields: { name, location, opening_time, closing_time }
File fields: { image (optional), aadhar_card (required), hall_approval_form (required) }
```
Status:
```
If successful:
  201 Registration submitted (returns canteen data)
else:
  400 Validation error
  403 Only managers can register canteens
  401 Not authenticated
```

f. Update Canteen Image

URL: `/api/canteens/<canteen_id>/image/`
Method: PATCH (multipart/form-data)
Data: `{ image }` (file upload)
Status:
```
```

If successful:
  200 Returns updated canteen data
else:
  400 No image file provided
  403 Only managers can update canteen image
  404 Canteen not found
```

**g. Update Canteen Status**

URL: `/api/canteens/<canteen_id>/status/`
Method: PATCH
Data: `{ status }` (OPEN, BUSY, CLOSED, etc.)
Status:
```
If successful:
  200 Returns updated canteen data
else:
  400 Invalid status transition
  403 Only managers can update status
  404 Canteen not found
```

**h. Update Canteen Timings**

URL: `/api/canteens/<canteen_id>/timings/`
Method: PATCH
Data: `{ opening_time, closing_time }` (HH:MM format, at least one required)
Status:
```
If successful:
  200 Returns updated canteen data
else:
  400 Invalid time format / No times provided
  403 Only managers can update timings
  404 Canteen not found
```
```

i. View Menu

URL: `/api/canteens/<canteen_id>/menu/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of dishes
else:
  404 Canteen not found
```

j. Popular Dishes (Per Canteen)

URL: `/api/canteens/<canteen_id>/menu/popular/`
Method: GET
Query Params: `limit` (default 20), `category` (optional), `available_only` (default true)
Status:
```
If successful:
  200 Returns list of popular dishes sorted by rating
else:
  404 Canteen not found
```

k. Add Dish to Menu

URL: `/api/canteens/<canteen_id>/menu/add/`
Method: POST (multipart/form-data or JSON)
Data: `{ name, price, category, image (optional file) }`
Status:
```
If successful:
  201 Returns created dish data
else:
  400 Validation error
  403 Only managers can add dishes
```

```
  404 Canteen not found
```

l. Update / Delete Dish

URL: `/api/canteens/dishes/<dish_id>/`
Method: PATCH / DELETE (multipart/form-data for PATCH)
Data (PATCH): `{ name, price, category, image (optional file) }` (partial)
Status:
```
If successful:
  200 Returns updated dish data (PATCH)
  204 Dish deleted (DELETE)
else:
  403 Only managers can manage dishes
  404 Dish not found
```

m. Toggle Dish Availability

URL: `/api/canteens/dishes/<dish_id>/toggle/`
Method: POST
Data: None
Status:
```
If successful:
  200 Returns updated dish data
else:
  403 Only managers can toggle availability
  404 Dish not found
```

n. Popular Dishes (Global)

URL: `/api/canteens/dishes/popular/`
Method: GET
Query Params: `limit` (default 20, max 50), `category` (optional), `available_only` (default true)
Status:
```
```

If successful:
  200 Returns globally ranked list of popular dishes
```

**o. Manage Holidays**

URL: `/api/canteens/<canteen_id>/holidays/`
Method: GET / POST / DELETE
Data (POST): `{ date, reason }` · Data (DELETE): `{ date }` (YYYY-MM-DD format)
Status:
```
If successful:
  200 Returns list of holidays (GET) / Holiday removed (DELETE)
  201 Holiday added (POST)
else:
  400 Validation error / Date required
  403 Only managers can manage holidays
  404 Canteen not found
```

**p. View Documents**

URL: `/api/canteens/<canteen_id>/documents/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns document list with download URLs
else:
  403 Not authorized (manager of this canteen or admin only)
  404 Canteen not found
```

**q. Serve Document File**

URL: `/api/canteens/<canteen_id>/documents/<filename>/`
Method: GET
Data: None
```

Status:
```
If successful:
  200 Returns file download (FileResponse)
else:
  403 Not authorized
  404 Canteen or document not found
```

r. Manager Dashboard

URL: `/api/canteens/manager/dashboard/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns { canteen, earnings { total_revenue, completed_orders },
      queue { pending_orders, active_orders, estimated_wait_minutes } }
else:
  403 Only managers
  404 No canteen assigned
```

s. Manager Analytics

URL: `/api/canteens/manager/analytics/`
Method: GET
Query Params: `year` (optional, defaults to all)
Status:
```
If successful:
  200 Returns monthly revenue and order breakdown
else:
  400 Invalid year parameter
  403 Only managers can view analytics
  404 No canteen assigned
```

t. Manager Dish Analytics

URL: `/api/canteens/manager/dish-analytics/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns { dish_frequency, top_by_frequency, top_by_revenue }
      (last 30 days)
else:
  403 Only managers can view dish analytics
  404 No canteen assigned
```

u. Manager Monthly Revenue

URL: `/api/canteens/manager/monthly-revenue/`
Method: GET
Query Params: `year` (default current year), `month` (1–12, default current month)
Status:
```
If successful:
  200 Returns per-dish revenue breakdown for the specified month
else:
  400 Invalid year or month parameter
  403 Only managers can view monthly revenue
  404 No canteen assigned
```

Orders

a. Place Order

URL: `/api/orders/place/`
Method: POST
Data: `{ canteen_id, items: [{ dish_id, quantity }], wallet_pin, notes (optional), customer_name (optional), roll_no (optional) }`

Status:
```
If successful:
  201 Order placed successfully (returns order data)
else:
  400 Invalid PIN / Insufficient funds / Invalid items
  403 Only customers can place orders
  404 Canteen not found
```

b. Order Detail

URL: `/api/orders/<order_id>/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns order details (items, payment, status, wait time)
else:
  403 Not authorized (own order or canteen manager only)
  404 Order not found
```

c. Customer Order History

URL: `/api/orders/history/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of customer's orders
else:
  403 Only customers have order history
```

d. Detailed Order History

URL: `/api/orders/history/detailed/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns comprehensive order history with dish details,
      is_rated status, total price, and payment info
else:
  403 Only customers have order history
```

e. Previous Order

URL: `/api/orders/previous-order/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns most recent completed order with dish details
else:
  403 Only customers have order history
  404 No previous orders found
```

f. Pending Orders (Manager)

URL: `/api/orders/pending/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of pending orders for manager's canteen
else:
  403 Only managers can view pending orders
```

```
  404 No canteen assigned
```

g. Active Orders (Manager)

URL: `/api/orders/active/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of active orders being processed
else:
  403 Only managers can view active orders
  404 No canteen assigned
```

h. Accept Order

URL: `/api/orders/<order_id>/accept/`
Method: POST
Data: None
Status:
```
If successful:
  200 Order accepted (PENDING → ACCEPTED), returns order data
else:
  400 Invalid state transition
  403 Only managers can accept orders
  404 Order not found
```

i. Reject Order

URL: `/api/orders/<order_id>/reject/`
Method: POST
Data: `{ reason (optional) }`
Status:
```
```

```
If successful:
  200 Order rejected + auto-refund to customer wallet
else:
  400 Invalid state transition
  403 Only managers can reject orders
  404 Order not found
```

j. Mark Order Ready

```
URL: `/api/orders/<order_id>/ready/`
Method: POST
Data: None
Status:
```

```
If successful:
  200 Order marked as ready for pickup
else:
  400 Invalid state transition
  403 Only managers can mark orders ready
  404 Order not found
```

k. Mark Order Completed

```
URL: `/api/orders/<order_id>/complete/`
Method: POST
Data: None
Status:
```

```
If successful:
  200 Order completed (funds credited to manager wallet)
else:
  400 Invalid state transition
  403 Only managers can complete orders
  404 Order not found
```

l. Request Cancellation

URL: `/api/orders/<order_id>/cancel/`
Method: POST
Data: None
Status:
```
If successful:
  200 Cancellation requested — awaiting manager approval
else:
  400 Order cannot be cancelled in its current state
  403 Only customers can request cancellation
  404 Order not found
```

m. Approve Cancellation

URL: `/api/orders/<order_id>/approve-cancel/`
Method: POST
Data: None
Status:
```
If successful:
  200 Cancellation approved → CANCELLED → REFUNDED
else:
  400 Invalid state transition
  403 Only managers can approve cancellations
  404 Order not found
```

n. Reject Cancellation

URL: `/api/orders/<order_id>/reject-cancel/`
Method: POST
Data: `{ reason (optional) }`
Status:
```
If successful:
```

```
  200 Cancellation rejected, order reverts to previous state
else:
  400 Invalid state transition
  403 Only managers can reject cancellations
  404 Order not found
```

o. Rate Order

URL: `/api/orders/<order_id>/rate/`
Method: POST
Data: `{ ratings: [{ dish_id, rating (1–5) }, ...] }`
Status:
```
If successful:
  200 Order rated successfully
else:
  400 Invalid ratings / Duplicate dish_id / Already rated
  403 Only customers can rate orders
  404 Order not found
```

p. Manager Order History

URL: `/api/orders/manager-history/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns full order history for the manager's canteen
else:
  403 Only managers can view canteen order history
  404 No canteen assigned
```

q. Order Wait Time

URL: `/api/orders/<order_id>/wait-time/`

Method: GET
Data: None
Status:
```
If successful:
  200 Returns { order_id, status, estimated_wait_minutes }
else:
  403 Not authorized
  404 Order not found
```

Cake Reservations

a. Check Availability

URL: `/api/cakes/check-availability/`
Method: POST
Data: `{ canteen_id, date }`
Status:
```
If successful:
  200 Returns availability result (available/unavailable + reason)
else:
  404 Canteen not found
```

b. Submit Reservation

URL: `/api/cakes/reserve/`
Method: POST
Data: `{ canteen_id, flavor, size, design (optional), message (optional), pickup_date, pickup_time, advance_amount, wallet_pin }`
Status:
```
If successful:
  201 Reservation submitted — awaiting manager approval
else:
  400 Invalid PIN / Insufficient funds / Validation error
```

```
  403 Only customers can make reservations
  404 Canteen not found
```

c. My Reservations

URL: `/api/cakes/my-reservations/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of customer's cake reservations
else:
  403 Only customers have reservations
```

d. Pending Reservations (Manager)

URL: `/api/cakes/pending/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of pending reservations for manager's canteen
else:
  403 Only managers
  404 No canteen assigned
```

e. All Reservations (Manager)

URL: `/api/cakes/manager-all/`
Method: GET
Data: None
Status:
```
If successful:
```

```
  200 Returns all reservations for manager's canteen (all statuses)
else:
  403 Only managers
  404 No canteen assigned
```

f. Accept Reservation

URL: `/api/cakes/<reservation_id>/accept/`
Method: POST
Data: None
Status:
```
If successful:
  200 Reservation accepted (PENDING_APPROVAL → CONFIRMED)
else:
  400 Invalid state transition
  403 Only managers
  404 Reservation not found
```

g. Reject Reservation

URL: `/api/cakes/<reservation_id>/reject/`
Method: POST
Data: `{ reason (optional) }`
Status:
```
If successful:
  200 Reservation rejected + auto-refund to customer wallet
else:
  400 Invalid state transition
  403 Only managers
  404 Reservation not found
```

h. Complete Reservation

URL: `/api/cakes/<reservation_id>/complete/`

Method: POST
Data: None
Status:
```
If successful:
  200 Reservation completed (marked as picked up)
else:
  400 Invalid state transition
  403 Only managers
  404 Reservation not found
```

Administration

a. List Pending Canteen Requests

URL: `/api/admin/canteen-requests/`
Method: GET
Data: None
Status:
```
If successful:
  200 Returns list of canteens with UNDER_REVIEW status
      (includes document links)
else:
  403 Admin access required
```

b. Approve Canteen Registration

URL: `/api/admin/canteen-requests/<canteen_id>/approve/`
Method: POST
Data: None
Status:
```
If successful:
  200 Canteen approved and activated (manager marked as staff)
else:
```

```
  403 Admin access required
  404 Canteen request not found
```

c. Reject Canteen Registration

```
URL: `/api/admin/canteen-requests/<canteen_id>/reject/`
Method: POST
Data: `{ reason (optional) }`
Status:
```

If successful:

```
  200 Canteen rejected (rejection email sent to manager)
else:
  403 Admin access required
  404 Canteen request not found
```

d. List Pending Manager Registrations

```
URL: `/api/admin/manager-registrations/`
Method: GET
Data: None
Status:
```

If successful:

```
  200 Returns list of pending manager registrations
else:
  403 Admin access required
```

e. Approve Manager Registration

```
URL: `/api/admin/manager-registrations/<registration_id>/approve/`
Method: POST
Data: None
Status:
```

If successful:

```
  200 Manager approved and account created (approval email sent)
else:
  403 Admin access required
  404 Registration not found
```

f. Reject Manager Registration

URL: `/api/admin/manager-registrations/<registration_id>/reject/`
Method: POST
Data: `{ reason (optional) }`
Status:
```
If successful:
  200 Manager registration rejected (rejection email sent)
else:
  403 Admin access required
  404 Registration not found
```

3. Completeness

Implemented Features

The project includes a fully functional, centralised canteen automation and queue management platform complete with role-based authentication, extensive property (canteen) management, secure internal transactions, and robust real-time order lifecycle tracking. The application utilizes a powerful Django ORM backend paired with PostgreSQL/SQLite databases to ensure swift data retrieval and modular scalability. Below is the comprehensive list of all implemented features:

1. User Authentication & Security

- Secure authentication system with logic for generating and verifying One-Time Passwords (OTPs) for user registration and password recovery (`/api/auth/register/`, `/api/auth/verify-otp/`).
- Role-based access control maintaining separate, dedicated profiles for different user types: Customers (`CustomerProfile`), Managers (`CanteenManagerProfile`), and Administrators (`AdminProfile`).
- Unauthorized actions are strictly blocked by the system. For instance, a customer cannot access canteen management endpoints or admin document repositories.
- Secure PIN management (`/api/users/wallet/set-pin/`) utilizing cryptographic hashing to protect wallet transactions, ensuring that even if a physical device is compromised, financial actions require explicit user authorization.
- Login and logout session management functionalities implemented across all user roles (`/api/auth/login/`, `/api/auth/logout/`).

2. User Profile & Digital Wallet Management

- Comprehensive profile management allowing users to view and edit their specific demographic details, including integrated `hall` and `room_number` fields for precise localized data (`/api/users/profile/`).
- A fully functional, closed-loop SkipQ Wallet system. Users can seamlessly check their wallet balance (`/api/users/wallet/`) and dynamically add funds to their account (`/api/users/wallet/add-funds/`).
- Advanced transaction security through a user-defined Wallet PIN (`/api/users/wallet/set-pin/`). All financial transactions require this 4-6 digit hashed PIN, ensuring funds remain secure even if an active session is physically compromised.
- Favorite Dishes tracking via a Many-to-Many (`M2M`) database junction table, allowing users to bookmark preferred meals for rapid future reordering.

3. Canteen Onboarding & Menu Configuration

- Canteen managers can register their establishments by uploading necessary credentials, including canteen images and verification documents (`/api/canteens/register/`).
- Dynamic operational control allowing managers to update their current canteen status (Open, Closed, Busy, Emergency) and configure upcoming holidays (`/api/canteens/<id>/status/`, `/api/canteens/<id>/holidays/`).
- Extensive menu management tools enabling managers to add new dishes (`/api/canteens/<id>/menu/add/`), edit existing items, or seamlessly toggle item availability during operating hours (`/api/canteens/dishes/<id>/toggle/`).
- Uploaded dish photos are automatically processed and stored centrally on the filesystem as `.jpg` assets.
- The system automatically computes and retrieves popular dishes both globally and on a per-canteen basis (`/api/canteens/dishes/popular/`).

4. Real-Time Order Placement & Queue Management

- Customers can securely place orders, which prompts a mandatory PIN verification and atomic wallet fund deduction (`/api/orders/place/`).
- Managers utilize a dedicated dashboard to view both pending and active orders (`/api/orders/pending/`, `/api/orders/active/`).
- Comprehensive order lifecycle management: Managers can actively accept, mark as ready, or complete orders.
- Automated financial protections: If a manager rejects an incoming order, the system automatically triggers a refund to the customer's SkipQ wallet.
- A dedicated cancellation pipeline allowing customers to submit a cancellation request (`/api/orders/<id>/cancel/`), which the manager can either systematically approve or reject based on the preparation stage.
- The application provides an estimated waiting time for customers to help manage foot traffic efficiently.

5. Specialized Cake Reservation System

- A dedicated advance-booking architecture for event and birthday cakes.
- Custom, per-canteen `lead_time_config` parameters that strictly prevent customers from booking cakes within impossibly short timeframes.
- Upfront payment processing mechanisms tied directly to a specialized Pending Approval state (`PENDING_APPROVAL`, `CONFIRMED`), allowing bakery managers to carefully review and either accept or refund the customized cake orders.

6. Ratings and Statistical Analytics

- A streamlined post-order numerical rating system (`/api/orders/<id>/rate/`), allowing customers to evaluate completed orders.
- Algorithmic generation of "Popular Dishes," fetching trending items both globally across the platform and specifically within individual canteens (`/api/canteens/dishes/popular/`).
- Detailed manager analytics dashboards serving rich statistical insights into monthly total orders, total revenue, and itemized dish-performance distributions.

7. Administrative System Oversight

- A functional V1 Admin Portal enables system administrators to view pending canteen registration requests (`/api/admin/canteen-requests/`).
- Administrators can securely access and review the compliance documents uploaded by canteen managers (`/api/canteens/<id>/documents/`).
- Admins have the authoritative capability to approve or reject canteen requests, thereby updating their visibility status across the entire customer platform.

8. Secure and Centralized File Serving

- The backend securely partitions filesystem storage to separate public assets from sensitive documents.
- Publicly accessible static directories serve canteen images (`files/canteen_images/`) and menu dish photos (`files/dish_images/`).
- A strictly protected directory structure handles sensitive admin documents (`files/documents/`), preventing unauthorized public access to canteen compliance files.

Future Development Plans

1. Live Payment Gateway Integration

Currently, the application relies on an internal, simulated SkipQ Wallet. In future iterations, we will transition from this fake wallet staging to a fully integrated live payment gateway (such as Razorpay or direct UPI APIs), allowing users to process financial transactions directly from their bank accounts to the canteen ecosystem.

2. Machine Learning-Powered Wait Time Predictions

The current software estimates wait times using a robust but static calculation logic (e.g., standard time multiplied by active order count). Moving forward, we plan to implement a Machine Learning inference pipeline that ingests historical order data, canteen load, and time-of-day metrics to predict highly accurate, dynamic wait times on the fly.

3. Advanced Email Services and Notification Dispatch

While the internal logic and token generation for OTPs and rejection notifications are fully implemented, the active dispatch relies on future SMTP/SendGrid configuration. We plan to fully configure the email backend to dispatch automated emails for new user verification, forgotten password resets, and formal canteen rejection notices for managers.

4. Expanded User Demographics and Deliveries

Currently, the customer profile captures basic identification. We wish to expand the demographic scope by migrating the database to include specific student details. This foundational data will pave the way for a future localized hostel delivery network.

5. Database Optimization and Migration Cleanup

To ensure the system remains highly maintainable and clean, future development will include strict migration sweeps to drop deprecated database columns (such as the legacy `aadhar_card` and `hall_approval_form` fields from the primary models, as well as dead `review_text` strings), optimizing the database to strictly match the filesystem-based asset management and rating-only feedback systems.

Appendix A - Group Log

S.No.	Date	Time	Venue	Agenda / Key Activity	Type
1	2026-03-12	21:00 - 22:30	RM Building	Setup of PostgreSQL production environment and migration strategy.	Offline
2	2026-03-16	22:00 - 23:30	Google Meet	Frontend-Backend integration review for Order Lifecycle Management.	Online
3	2026-03-20	21:30 - 23:00	RM Building	Testing session for Secure Wallet PIN verification and atomic transactions.	Offline
4	2026-03-24	22:00 - 00:00	RM Building	Final bug scrubbing and preparation of live deployment links.	Offline